

6_Practicals Adversarial Machine Learning in Security Analytics

Step 1: Import libraries.

```
#import all necessary libraries
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris

import numpy as np
from matplotlib import pyplot as plt

from art.estimators.classification import SklearnClassifier
from art.attacks.evasion import ZooAttack
from art.utils import load_mnist

import warnings
warnings.filterwarnings('ignore')
```

Step 2: Load the dataset.

```
# set x_train, y_test, y_train and y_test
(x_train, y_train), (x_test, y_test), min_, max_ = load_mnist() #load
the MNIST dataset
```

```
x_train.shape #check the shape of the training features
```

```
(60000, 28, 28, 1)
```

```
x_test.shape #check the shape of the testing features
```

```
(10000, 28, 28, 1)
```

```
y_train.shape #check the shape of y_train
```

```
(60000, 10)
```

```
y_test.shape #check the shape of the testing labels
```

```
(10000, 10)
```

Step 3: Compute Features and Samples.

```
n_samples_train = x_train.shape[0] #calculate the number of samples for
the training dataset by taking 0th position from x_train.shape ([60000,
28, 28, 1]), which 60000
n_features_train = x_train.shape[1] * x_train.shape[2] *
x_train.shape[3] #compute the number of training features from
x_train.shape ([60000, 28, 28, 1]), which multiplies 28*28*1
n_samples_test = x_test.shape[0] #calculate the number of samples for
the testing dataset by taking 0th position from x_test.shape ([10000,
28, 28, 1]), which 10000
n_features_test = x_test.shape[1] * x_test.shape[2] * x_test.shape[3]
#compute the number of testing features from x_test.shape ([10000, 28,
28, 1]), which multiplies 28*28*1
```

```
n_samples_train #print the number of samples for training datasets
```

```
60000
```

```
n_features_train #print the number of features for training dataset
```

```
784
```

```
n_samples_test
```

```
10000
```

```
n_features_test #print the number of features for testing dataset
```

```
784
```

```
y_train #printing y_train
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [1., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 1., 0.]], dtype=float32)
```

```
y_test #printing y_test
```

```
array([[0., 0., 0., ..., 1., 0., 0.],
       [0., 0., 1., ..., 0., 0., 0.],
       [0., 1., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]], dtype=float32)
```

Step 4: Reshape and Convert Labels.

```
x_train = x_train.reshape(n_samples_train, n_features_train) #reshaping
x_train from [60000, 28, 28, 1] to [60000, 784]
x_test = x_test.reshape(n_samples_test, n_features_test) #reshaping
x_testing from [10000, 28, 28, 1] to [10000, 784]
```

```
y_train = np.argmax(y_train, axis=1) # extrcating the index position of
the highest from each row to get training labels
y_test = np.argmax(y_test, axis=1) # extrcating the index position of
the highest from each row to get training labels
```

```
x_train.shape #printing the shape of the training features
```

```
➤ (60000, 784)
```

```
x_train #printing x_train
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

```
x_test.shape #printing shape of x_test
```

```
(10000, 784)
```

```
x_test #printing x_test
```

```
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

```
y_train.shape #printing y_train shape
```

```
(10000,)
```

```
y_test.shape #printing y_test shape
```

```
(10000,)
```

```
y_test #prinitng y_test
```

```
array([7, 2, 1, ..., 4, 5, 6])
```

Step 5: Subset the Dataset.

```
n_samples_max = 200 #set sample size of 200
x_train = x_train[0:n_samples_max] #get 200 samples from x_train
y_train = y_train[0:n_samples_max] #get 200 of the respective labels
for those 200 x_train samples
x_test = x_test[0:n_samples_max] #get 200 samples from x_test
y_test = y_test[0:n_samples_max] #get 200 of the respective labels for
those 200 x_test samples
```

```
x_train.shape #printing the shape of x_train
(200, 784)
```

```
y_train.shape #printing the shape of y_train
(200,)
```

```
x_test.shape #printing the shape of x_test
(200, 784)
```

```
y_test.shape #printing the shape of y_test
(200,)
```

Step 6: Train a classifier: Random Forest Classifier.

```
#defining the random forest classifier
model = RandomForestClassifier(n_estimators=10, criterion='gini',
max_depth=None, min_samples_split=2,
                                min_samples_leaf=1,
min_weight_fraction_leaf=0.0, max_features=None,
                                max_leaf_nodes=None,
min_impurity_decrease=0.0,
                                bootstrap=True, oob_score=False,
n_jobs=None, random_state=None, verbose=0,
                                warm_start=False, class_weight=None)
```

```
model.fit(x_train, y_train) #fit the model x_train and y_train
```

```
RandomForestClassifier
RandomForestClassifier(max_features=None, n_estimators=10)
```

Step 7: Generate Adversarial Samples Using ZOO Attack.

```
# Construct a DT classifier to use it as a parameter for Zoo package
# model was already created using RandomForestClassifier()
art_classifier = SklearnClassifier(model=model)
```

```
art_classifier #print the art classifier
```

```
ScikitlearnRandomForestClassifier(model=RandomForestClassifier(max_features=None, n_estimators=10,
std=1.0, apply_fit=True, apply_predict=True), preprocessing_defences=None, postprocessing_defence
std=1.0, apply_fit=True, apply_predict=True)], nb_classes=10, input_shape=(784,)), use_logits=False
```

```
# define the zoo attack
```

```
zoo = ZooAttack(classifier=art_classifier, confidence=0.0,
targeted=False, learning_rate=1e-1, max_iter=100,
                binary_search_steps=20, initial_const=1e-3,
abort_early=True, use_resize=False,
                use_importance=False, nb_parallel=10, batch_size=1,
variable_h=0.25)
```

```
# generate the adversarial samples using x_train and store it in
x_train_adv
```

```
x_train_adv = zoo.generate(x_train)
```

```
ZOO: 100% 200/200 [08:49<00:00, 2.43s/it]
```

```
#generate the adversarial samples using x_test and store it in
x_test_adv
```

```
x_test_adv = zoo.generate(x_test)
```

```
ZOO: 100% 200/200 [06:20<00:00, 1.81s/it]
```

```
#training_the_model to with normal x_train and y_train
```

```
model_testing.fit(x_train, y_train)
```

RandomForestClassifier

```
RandomForestClassifier(max_features=None, n_estimators=10)
```

Step 8: Train and evaluate the model using the normal training data and the normal test data.

```
#import the necessary libraries to print confusion matrix and
classiifcation report
from sklearn.metrics import confusion_matrix, classification_report
y_pred = model_testing.predict(x_test) #predict the values using x_test
and store in y_pred
print(y_pred) #print y_pred
print(confusion_matrix(y_test, y_pred)) #print confusion matrix, use
y_pred and y_test
print(classification_report(y_test, y_pred)) #print classification
report, use y_pred and y_test
```

```
[ 7 2 1 0 4 1 4 1 2 9 0 2 9 0 1 7 3 7 2 4 9 6 1 0 4 0 7 4 0 1 3 1 3 2 7 7 7
 1 1 1 1 7 4 2 3 3 1 2 4 4 6 3 4 4 0 9 4 1 9 5 7 2 4 3 9 9 0 4 3 0 7 0 0 8
 1 1 3 7 9 7 7 6 2 7 8 4 7 3 6 1 0 6 9 3 1 9 1 1 6 3 1 0 3 4 3 7 8 1 3 7 8
 1 2 9 7 4 4 4 9 7 3 1 2 6 9 1 2 1 8 0 6 6 3 7 8 1 8 1 4 9 6 7 3 1 7 1 9 0
 8 1 3 2 0 3 1 3 4 9 7 6 4 2 0 4 4 3 4 4 1 4 4 5 2 0 2 2 1 4 1 8 1 8 3 0 8
 9 1 3 0 1 1 1 4 3 0 4 1 6 4 2]
[[11  0  1  0  1  0  0  0  2  2]
 [ 0 28  0  0  0  0  0  0  0  0]
 [ 2  3  7  0  0  0  0  3  1  0]
 [ 2  0  2 10  1  0  0  1  0  0]
 [ 0  1  1  0 21  0  1  1  0  3]
 [ 4  1  1  9  3  1  0  1  0  0]
 [ 2  2  2  0  3  0 11  0  0  0]
 [ 0  3  2  0  0  1  0 16  0  2]
 [ 0  0  1  0  1  0  0  0  7  1]
 [ 0  2  1  6  1  0  0  1  1  9]]
      precision    recall  f1-score   support

    0         0.52      0.65      0.58         17
    1         0.70      1.00      0.82         28
    2         0.39      0.44      0.41         16
    3         0.40      0.62      0.49         16
    4         0.68      0.75      0.71         28
    5         0.50      0.05      0.09         20
    6         0.92      0.55      0.69         20
    7         0.70      0.67      0.68         24
    8         0.64      0.70      0.67         10
    9         0.53      0.43      0.47         21

 accuracy          0.60         200
 macro avg         0.60         200
 weighted avg      0.61         200
```

```
#training_the_model to with normal x_train_adv and y_train
model_testing.fit(x_train_adv, y_train)
```

▼ RandomForestClassifier ⓘ ?

RandomForestClassifier(max_features=None, n_estimators=10)

Step 9: Train and test the model using adversarial training sample data and adversarial test sample data.

```
#import the necessary libraries to print confusion matrix and
classiifcation report
from sklearn.metrics import confusion_matrix, classification_report
y_pred = model_testing.predict(x_test_adv) #predict the values using
the adversarial samples generated from x_test_adv, and store in y_pred
print(y_pred) #print the predictions
print(confusion_matrix(y_test, y_pred)) #print confusion matrix, use
y_pred and y_test
print(classification_report(y_test, y_pred)) #print classification
report, use y_pred and y_test
```

```
[7 2 1 0 4 1 4 3 2 9 0 4 9 0 1 0 3 7 2 4 9 4 4 4 3 0 7 4 0 1 3 1 3 0 9 2 7
 1 1 1 1 7 4 1 3 3 1 4 4 4 6 2 4 3 2 0 4 1 4 5 7 8 3 9 3 4 8 4 3 0 7 0 0 8
 1 4 3 7 9 7 9 6 0 7 9 4 7 3 6 1 3 6 9 5 1 4 1 1 6 3 4 0 7 4 3 7 2 1 9 4 8
 1 1 9 7 9 9 4 4 6 3 4 7 6 9 9 0 5 8 9 6 6 5 2 8 1 0 1 6 4 6 7 3 1 7 1 8 4
 0 1 3 3 9 9 1 9 4 0 0 1 4 6 5 4 6 3 0 1 1 4 4 7 2 0 2 7 1 3 1 8 1 1 9 0 2
 9 2 3 0 1 1 1 0 3 0 7 1 6 9 8]
```

	precision	recall	f1-score	support
0	0.71	1.00	0.83	17
1	0.76	1.00	0.86	28
2	0.50	0.38	0.43	16
3	0.33	0.50	0.40	16
4	0.64	0.75	0.69	28
5	0.80	0.20	0.32	20
6	0.93	0.65	0.76	20
7	0.80	0.67	0.73	24
8	0.67	0.60	0.63	10
9	0.41	0.43	0.42	21
accuracy			0.64	200
macro avg	0.65	0.62	0.61	200
weighted avg	0.67	0.64	0.63	200